Machine Learning Classification of Player Feedback from Steam Reviews

Student: Robert Mayfield **Project:** Udacity AI Masters Capstone — Applied Machine Learning

---

Overview

This project trains and evaluates a supervised binary text classifier that predicts whether a player review reflects a positive or negative experience. The classifier takes raw review text as input and produces a single binary output: positive (1) or negative (0). The primary intended application is playtesting feedback analysis during game development, where teams receive large volumes of written tester responses and need a reliable automated first pass to surface negative signals before manual review.

The training data is the Steam Reviews Dataset (forgemaster, 2021), a publicly available collection of 15,437,471 player reviews labeled with a binary recommendation signal (`voted_up`), available at https://www.kaggle.com/datasets/forgemaster/steam-reviews-dataset. Because Steam reviews represent the largest public source of labeled player generated text in natural language, they provide a strong proxy for training a general purpose player feedback classifier, even though playtesting notes differ in register and specificity from consumer reviews.

The workflow covers data loading, quality review, text preprocessing, TF-IDF feature extraction, comparative evaluation of four candidate classifiers, hyperparameter tuning, error analysis, and post-training fairness evaluation using the IBM AIF360 toolkit. The trained classifier and vectorizer are saved as reusable artifacts for integration into downstream pipelines.

Dataset Description

Source: Steam Reviews Dataset (forgemaster, 2021), accessed via Kaggle. The full dataset comprises 11 CSV files totaling approximately 5.32 GB and 15,437,471 reviews across 8,183 games.

Sampling: A stratified random sample of approximately 100,000 rows (99,999 after sampling) was drawn from the full archive. After quality filtering, 81,805 reviews were retained for modeling.

Target variable: `voted_up` (boolean). A value of True indicates the reviewer recommends the game; False indicates they do not. This label is provided directly by the reviewer at the time of posting, making it a clean binary supervised learning target.

Class distribution: The dataset is strongly class imbalanced. After filtering, 69,749 reviews (85.3%) are positive and 12,056 (14.7%) are negative, a ratio of roughly 5.8 to 1. This imbalance reflects real platform behavior on Steam, where the majority of reviews are recommendations. A naive classifier that always predicts positive would achieve 85.3% accuracy without learning any meaningful signal, which is why macro F1 score is used as the primary evaluation criterion rather than raw accuracy.

Key quality findings from inspection:

- 16,853 reviews (16.9%) contained fewer than 3 words and were removed. These reviews provide near empty TF-IDF vectors and almost no predictive signal.

- 10,800 reviews (10.8%) were exact text duplicates. After the word count filter, 1,341 remaining duplicates were dropped to prevent the same text from appearing in both training and test sets.
- 31 reviews (0.03%) contained HTML tags; 557 (0.56%) contained URLs. Both were stripped during preprocessing.
- 622 reviews (0.6%) exceeded a 30% non-ASCII character ratio, including emoji heavy text, Braille block art, and non English reviews. Non-ASCII characters were removed during normalization.

Modeling Approach

This project uses a pipeline of TF-IDF vectorization followed by a linear classifier. Raw review text is first cleaned by `preprocess_text()`, then converted to a 50,000-feature TF-IDF matrix by `build_feature_matrix()`. Four scikit-learn classifiers were trained and compared on the same feature representation: Logistic Regression, Linear SVM, Multinomial Naive Bayes, and Complement Naive Bayes. The selected model, Linear SVM with `class_weight='balanced'`, feeds directly from the TF-IDF matrix without any intermediate embedding or dimensionality reduction step. Both helper functions are defined in `src/preprocessing.py` and imported at the top of the notebook; model evaluation is handled by `evaluate_model()` in `src/evaluation.py`.

Preprocessing Decisions

Text preprocessing is handled by `preprocess_text()`, defined in `src/preprocessing.py`. The function applies the following steps in order: lowercase conversion, HTML tag removal, BBCode markup removal, URL removal, non-ASCII character stripping, punctuation removal, and whitespace normalization. These steps address all noise categories identified in the quality review.

The minimum word count threshold of 3 words was chosen as the lowest defensible threshold for TF-IDF signal. Raising it to 5 words would remove 26.6% of the dataset; the 3 word minimum minimizes data loss while excluding reviews with no meaningful textual content.

Feature extraction is handled by `build_feature_matrix()`, also in `src/preprocessing.py`. This function vectorizes the cleaned review text using TF-IDF with the following settings: 50,000 features, unigrams and bigrams, minimum document frequency of 5, maximum document frequency of 95%, and sublinear TF scaling. Sublinear scaling applies $\log(1 + \text{tf})$ to reduce the weight of very high frequency terms, which is standard practice for text classification tasks (Manning et al., 2008). Bigrams capture short phrases such as “not good” and “worth buying” that carry sentiment signal not recoverable from individual tokens.

The vectorizer was fit on the training set only and applied to the test set without refitting, preserving a clean held out evaluation.

Model Selection and Metric Justification

Four candidate models were trained and compared on the same 80/20 stratified train/test split. All four were linear or probabilistic classifiers, which are appropriate for high dimensional sparse

TF-IDF feature matrices. Tree based models such as Random Forest were excluded because they are computationally inefficient for sparse input where most feature values are zero (Pang & Lee, 2008).

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	Training Time
Linear SVM	0.9059	0.8064	0.8470	**0.8245**	1.917s
Logistic Regression	0.8936	0.7847	0.8717	0.8175	0.792s
Complement NB	0.8889	0.7764	0.8230	0.7965	0.027s
Multinomial NB	0.8789	0.8847	0.5995	0.6322	0.031s

Metric justification. Macro F1 score is the primary selection criterion because it computes the unweighted average of per class F1 scores, treating both the positive class and the minority negative class as equally important (Sokolova & Lapalme, 2009). Given the 5.8:1 class imbalance in this dataset, accuracy alone would reward a model for correctly predicting the majority class while ignoring failures on the negative class. Macro F1 prevents this by requiring the model to perform well on both classes to achieve a high score. Multinomial NB illustrates this failure mode directly: it achieved 87.9% accuracy but only 0.6322 macro F1, reflecting extremely low recall (0.600) on the negative class.

Selected model: Linear SVM (C=1.0, class_weight='balanced'). Linear SVM achieved the highest macro F1 (0.8245) and the highest accuracy (90.59%) across all four candidates. Class balanced weighting was applied to both Linear SVM and Logistic Regression, which rescales each class's contribution to the loss function in inverse proportion to its frequency, counteracting the effect of the imbalance during training.

Hyperparameter tuning. A 5 fold cross validated grid search over C values of {0.01, 0.1, 1.0, 10.0, 100.0} confirmed that C=1.0 is the optimal regularization value. The cross validated macro F1 peaked at 0.8237 at C=1.0 and declined at both smaller and larger values, indicating that C=1.0 sits at the optimal bias-variance tradeoff point for this dataset and feature space. Test set metrics were identical between the default and tuned models.

Results

The final model is Linear SVM with C=1.0, class balanced weighting, and TF-IDF features (50,000 features, unigrams and bigrams, sublinear TF scaling).

Summary metrics on the held out test set (16,361 reviews):

Metric	Value
Accuracy	90.59%
Macro Precision	0.8064
Macro Recall	0.8470
Macro F1	0.8245

Per-class breakdown:

Class	Precision	Recall	F1
Negative (0)	0.655	0.764	0.705
Positive (1)	0.958	0.930	0.944

The model performs substantially better on the positive class, which has roughly 5.8 times as many training examples. Negative recall of 0.764 means the model correctly identifies 76.4% of actual negative reviews. The remaining 23.6% of negative reviews are misclassified as positive, representing the primary failure mode.

Confusion matrix (test set):

	Predicted Negative	Predicted Positive
True Negative	1,841	570
True Positive	970	12,980

The 570 false positives are negative reviews the model predicted as positive. The 970 false negatives are positive reviews the model predicted as negative. False negatives outnumber false positives by nearly 2:1, which is expected given the class distribution.

Naive baseline comparison. A classifier that always predicts positive would achieve 85.3% accuracy. Linear SVM at 90.59% is 5.3 percentage points above that floor, and its macro F1 of 0.8245 reflects meaningful performance on both classes.

Interpretation for a Non Technical Audience

Imagine reading thousands of player reviews and trying to mark each one as a thumbs-up or thumbs-down. For a small number of reviews, a person can do this quickly, but at hundreds or thousands of reviews per session, it becomes impractical. This classifier automates that task.

The model was trained on historical Steam reviews, where each review already had a thumbs-up or thumbs-down label applied by the reviewer. It learned which words and short phrases tend to appear in positive reviews versus negative ones. When given a new review it has never seen, it uses those learned patterns to make a prediction.

The model is correct about 91% of the time overall. When it makes a mistake, it is roughly twice as likely to misread a positive review as negative as it is to misread a negative review as positive. This means in practice the model is slightly conservative: it will occasionally flag a positive piece of feedback as a concern, but it is less likely to overlook a genuine negative response.

The model works best on reviews of moderate length with clear, direct language. It struggles with short reviews that rely on sarcasm or irony, and with longer reviews that open with complaints before concluding positively. Any workflow using this classifier should treat its output as a first pass filter rather than a final judgment.

Error Analysis

The model made 1,540 errors on the 16,361 row test set, for an overall error rate of 9.4%. False negatives (970) outnumbered false positives (570) by nearly 2:1.

Error rate generally increased with review length, from 8.2% for reviews of 3 to 9 words up to 13.1% for reviews of 200 or more words. Errors averaged 71.5 words versus 54.3 words for correctly classified reviews. This pattern reflects a structural limitation of bag of words modeling: longer reviews tend to contain more nuanced or mixed language, and aggregating token frequencies cannot resolve contradictions within a single document.

False positive patterns (negative reviews predicted positive). Two recurring patterns appeared in the false positive sample. The first is negative framing used in service of a positive judgment: reviews such as "This review may not be positive, but..." contain strong negative surface tokens that outweigh the author's concluding sentiment. The second is short, colloquial reviews where tone cannot be recovered from token frequencies alone, for example "I got bored sooo not gud game," where "gud game" carries positive weight and overpowers the negative framing.

False negative patterns (positive reviews predicted negative). Several positive reviews opened with extended complaints or caveats before affirming the game, such as "I recommend you wait... Good game. Needs time for most of the bugs to be fixed." Because TF-IDF aggregates token weights without sentence context, the negative phrasing in the opening can outweigh the positive conclusion. Reviews structured as mixed critiques with a positive bottom line are consistently difficult for the model to classify correctly. These patterns are consistent with known limitations of bag of words approaches to sentiment analysis (Pang & Lee, 2008).

Limitations and Potential Bias

Bag of words representation. The TF-IDF model treats each review as an unordered set of token frequencies. It has no awareness of sentence structure, negation, word order, or context. A sentence like "not bad at all" is represented by the same feature weights as "bad at all not." The error analysis in Section 10 of the notebook demonstrates the practical impact of this limitation: reviews with negative openings and positive conclusions, and reviews that use ironic or sarcastic phrasing, are the dominant failure modes.

Test set used for model selection. Model selection compared four classifiers on the held-out test set; strictly, cross-validation on the training set should be used for model selection to preserve the held-out guarantee (Hastie, Tibshirani, & Friedman, 2009). The reported test set metrics should therefore be interpreted as slightly optimistic estimates of generalization performance.

Language coverage. The model was trained on English language text. Non English characters were stripped during preprocessing. Predictions on non English reviews should not be trusted, as the preprocessing pipeline removes most of the signal from non-ASCII text.

Domain shift. The training data consists of post launch consumer reviews written for a public audience on a gaming platform. Playtesting notes, the primary intended application, are typically written in a more technical, task focused register and may reference build specific features by name. A model trained on consumer reviews may underperform on playtesting feedback until fine tuned on labeled playtesting data.

Class imbalance and short review bias. The dataset is 85.3% positive, and short reviews are even more skewed: reviews under 25 words are 89.8% positive versus 80.2% for longer reviews. This subgroup label disparity means the model has less exposure to negative signal from short reviews during training. The post training bias analysis (Section 11 of the notebook) using AIF360 showed that all fairness metrics fall within accepted thresholds, but the false positive rate on short negative reviews (29.5%) is notably higher than for long negative reviews (19.7%).

Responsible Use

This classifier is designed as an advisory signal and should not be used as the sole basis for any decision that affects individual users. The false positive rate on short negative reviews means the model will misread a meaningful share of brief complaints as endorsements. Any deployment in a feedback triage context should include a review length flag so that downstream reviewers can apply appropriate scrutiny to short review predictions.

The class balanced weighting applied during training was the primary mitigation step taken to reduce bias. It prevents the model from learning to predict positive by default and was confirmed to contribute to near equal true positive recovery rates across the short and long review subgroups (equal opportunity difference of +0.0133, within the accepted threshold of 0.1).

Post training fairness evaluation using the IBM AIF360 toolkit (Bellamy et al., 2019) measured prediction fairness across review length subgroups (short: 3 to 24 words; long: 25 or more words). All four metrics fell within commonly accepted thresholds:

Metric	Value	Threshold
Statistical Parity Difference	+0.0866	within ±0.1
Equal Opportunity Difference	+0.0133	within ±0.1
Average Odds Difference	+0.0555	within ±0.1
Disparate Impact	1.1109	above 0.8

The statistical parity difference of +0.0866 reflects the underlying label distribution rather than model introduced bias: short reviews are inherently more positive in the data, so the model predicts positive more often for that group. The near zero equal opportunity difference confirms the model does not systematically fail to recover true positive labels from either subgroup.

Future Integrations

The trained classifier and TF-IDF vectorizer are saved as reusable artifacts (`review_classifier.pkl`, `tfidf_vectorizer.pkl`, `label_mapping.json`). Any system that can pass review text to the model can receive a binary sentiment signal without retraining.

Playtesting feedback analysis. The primary intended application is scoring written feedback from playtesting sessions during game development. Running tester responses through the classifier gives development teams an immediate sentiment distribution across a session, allowing designers to surface and prioritize negative feedback before a manual review cycle begins. Tracking sentiment

trends across multiple builds provides a lightweight signal for whether a change improved or degraded player response.

Real-time post launch monitoring. The classifier can be integrated into a pipeline that scores incoming reviews as they are posted, enabling teams to detect sentiment shifts within hours of a patch rather than waiting for aggregate rating scores to move.

Automated triage and routing. In support or community management workflows, negative predictions can be automatically routed to a human reviewer queue. Positive predictions can be aggregated for marketing or community highlights, reducing the manual effort of reading every review at scale.

Cross platform extension. The preprocessing pipeline and model architecture are not specific to Steam. The same approach can be applied to reviews from the App Store, Google Play, or Metacritic by retraining on platform specific labeled data, enabling sentiment signals from multiple sources to be compared on a consistent scale.

Agentic and automated reporting. The classifier is well suited as a component in an agentic pipeline where scheduled inference runs, trend aggregation, and structured report generation are automated without human intervention.

The evaluation framework developed here, including macro F1 as the primary criterion, AIF360 fairness metrics, and error analysis by review length group, provides a concrete benchmark against which any future model replacement can be measured.

References

Bellamy, R. K. E., Dey, K., Hind, M., Hoffman, S. C., Houde, S., Kannan, K., Lohia, P., Martino, J., Mehta, S., Mojsilovic, A., Nagar, S., Ramamurthy, K. N., Richards, J., Saha, D., Sattigeri, P., Singh, M., Varshney, K. R., & Zhang, Y. (2019). AI Fairness 360: An extensible toolkit for detecting and mitigating algorithmic bias. *IBM Journal of Research and Development, 63*(4/5), 4:1-4:15.

forgemaster. (2021). *Steam reviews dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/forgemaster/steam-reviews-dataset

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval, 2*(1-2), 1-135.

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing and Management, 45*(4), 427-437.